M4 Project 2: BST and AVL

Name: Vincent Goldberg

Course: C343 Data Structures

Date: June 21, 2026

Overview

This project implements two tree data structures and compares their behavior:

- A **Binary Search Tree (BST)** with `insert`, `delete`, `search`, the three traversals (in-order, pre-order, post-order), and `height`.
- A self-balancing **AVL Tree** that extends the BST and adds rotation-based rebalancing (left, right, left-right, right-left) to keep its height logarithmic after every insert and delete.

All operations are written recursively. Each public method delegates to a private helper that takes and returns a `Node`, which lets `AVLTree` override `insert`/`delete` and rebalance on the way back up the recursion.

The program (`Main.java`) runs Tasks 2, 4, and 5 in order and prints labeled results. Because the full traversal output is very large (512 values printed three times per tree), the complete console output is captured in `output.txt` and included as an appendix. The screenshots below show the relevant sections.

Part 1 — BST (Tasks 1 & 2)

A `BinarySearchTree` was built by inserting 512 unique random values in the range [1, 1024]. The program prints the tree height followed by all three traversals, each clearly labeled.

The in-order traversal prints the values in ascending sorted order, which confirms the tree is structured correctly.

```
===== Task 2: BST (512 random unique values) =====
Height: 19
In-order:   3 6 7 10 11 14 16 17 18 19 20 24 26 27 28 29 32 35 38 40 41 47 48 50 51 53 55 57 67 71 72 74 78 84
88 90 91 92 93 96 100 101 102 104 107 109 111 116 121 126 128 133 134 135 136 145 149 150 151 152 154 155 157
158 159 162 165 166 167 168 171 174 175 176 179 183 184 186 188 189 190 191 193 196 197 200 205 207 208 211 212
214 215 216 219 222 223 224 225 228 229 232 233 238 239 240 243 246 250 252 253 254 255 257 260 264 266 267 268
270 272 273 274 276 277 282 283 284 287 289 291 292 296 297 299 305 306 307 309 310 312 314 316 318 319 320 323
324 326 327 330 332 333 335 338 339 341 342 343 344 348 349 350 353 356 361 362 365 366 367 368 369 372 373 376
378 379 380 384 386 388 389 390 391 392 393 395 397 398 399 401 406 407 408 413 414 415 416 417 418 420 421 422
423 424 426 427 428 429 430 432 434 436 437 438 439 444 446 447 449 451 452 453 456 457 459 460 464 465 466 467
468 469 470 473 476 477 478 479 482 483 486 487 488 492 493 495 497 503 504 505 506 508 509 510 512 515 516 517
518 520 521 526 530 531 536 537 539 540 541 542 543 544 545 546 549 550 553 554 555 557 564 565 566 570 571 572
574 577 578 579 580 581 582 583 584 585 586 587 588 591 593 594 598 601 607 608 609 610 611 612 614 615 617 618
620 623 624 625 626 629 631 632 633 634 635 636 639 644 647 648 650 652 653 657 665 666 667 669 670 673 674 675
676 677 678 681 683 684 685 686 688 690 691 692 693 694 695 696 697 698 700 701 702 704 710 713 715 717 720 721
722 728 729 735 736 737 739 740 747 748 749 750 753 754 755 757 759 760 762 763 764 765 768 769 772 775 776 779
780 785 787 789 791 792 795 797 798 803 804 806 807 808 810 812 813 814 815 816 817 819 820 822 823 825 830 831
832 837 838 839 842 843 846 847 848 849 855 856 858 859 860 864 870 871 872 875 877 878 883 884 886 888 889 892
893 894 895 896 898 901 909 911 913 914 921 923 927 928 930 932 933 935 936 938 941 942 943 944 945 946 947 950
951 955 957 959 960 965 970 971 975 976 977 981 982 983 984 985 995 998 1001 1002 1004 1006 1011 1013 1015 1017
1020 1022 1024
```

Part 2 — AVL (Tasks 3 & 4)

The **same** 512 values from Task 2 were inserted into an `AVLTree`. The program again prints the height and all three traversals. The in-order traversal matches the BST's in-order output exactly (same sorted sequence), confirming both trees contain the identical data — but the AVL's height is noticeably smaller because it rebalances itself.

```
===== Task 4: AVL (same 512 values) =====
Height: 11
In-order:   3 6 7 10 11 14 16 17 18 19 20 24 26 27 28 29 32 35 38 40 41 47 48 50 51 53 55 57 67 71 72 74 78 84
88 90 91 92 93 96 100 101 102 104 107 109 111 116 121 126 128 133 134 135 136 145 149 150 151 152 154 155 157
158 159 162 165 166 167 168 171 174 175 176 179 183 184 186 188 189 190 191 193 196 197 200 205 207 208 211 212
214 215 216 219 222 223 224 225 228 229 232 233 238 239 240 243 246 250 252 253 254 255 257 260 264 266 267 268
270 272 273 274 276 277 282 283 284 287 289 291 292 296 297 299 305 306 307 309 310 312 314 316 318 319 320 323
324 326 327 330 332 333 335 338 339 341 342 343 344 348 349 350 353 356 361 362 365 366 367 368 369 372 373 376
378 379 380 384 386 388 389 390 391 392 393 395 397 398 399 401 406 407 408 413 414 415 416 417 418 420 421 422
423 424 426 427 428 429 430 432 434 436 437 438 439 444 446 447 449 451 452 453 456 457 459 460 464 465 466 467
468 469 470 473 476 477 478 479 482 483 486 487 488 492 493 495 497 503 504 505 506 508 509 510 512 515 516 517
518 520 521 526 530 531 536 537 539 540 541 542 543 544 545 546 549 550 553 554 555 557 564 565 566 570 571 572
574 577 578 579 580 581 582 583 584 585 586 587 588 591 593 594 598 601 607 608 609 610 611 612 614 615 617 618
620 623 624 625 626 629 631 632 633 634 635 636 639 644 647 648 650 652 653 657 665 666 667 669 670 673 674 675
676 677 678 681 683 684 685 686 688 690 691 692 693 694 695 696 697 698 700 701 702 704 710 713 715 717 720 721
722 728 729 735 736 737 739 740 747 748 749 750 753 754 755 757 759 760 762 763 764 765 768 769 772 775 776 779
780 785 787 789 791 792 795 797 798 803 804 806 807 808 810 812 813 814 815 816 817 819 820 822 823 825 830 831
832 837 838 839 842 843 846 847 848 849 855 856 858 859 860 864 870 871 872 875 877 878 883 884 886 888 889 892
893 894 895 896 898 901 909 911 913 914 921 923 927 928 930 932 933 935 936 938 941 942 943 944 945 946 947 950
951 955 957 959 960 965 970 971 975 976 977 981 982 983 984 985 995 998 1001 1002 1004 1006 1011 1013 1015 1017
1020 1022 1024
```

Part 3 — Comparative Testing & Discussion (Task 5)

Test datasets

Three datasets of 512 values each were inserted into both a BST and an AVL Tree, and the resulting heights were compared:

1. **Sorted ascending** (1, 2, 3, … 512)
2. **Sorted descending** (512, 511, … 1)
3. **Random unique** values drawn from [1, 1024]

Results

```
===== Task 5: BST vs AVL height comparison =====
Sorted ascending (1...512)   | n=512  | BST height=512  | AVL height=10
Sorted descending (512...1)  | n=512  | BST height=512  | AVL height=10
Random unique (1...1024)     | n=512  | BST height=17   | AVL height=11
```

(BST height for the random case varies per run since the data is randomized; the AVL height stays at roughly 10–11 regardless.)

Analysis

The experiment shows the difference between the two structures. A plain BST's shape — and therefore its performance — depends entirely on the order in which values are inserted, while an AVL Tree

guarantees a balanced shape regardless of insertion order.

Sorted input is the BST's worst case. When values arrive in ascending or descending order, every new value is larger (or smaller) than everything already in the tree, so each insert travels down the same side. The tree never branches and degenerates into a straight line — essentially a linked list — of height equal to the number of nodes (512). In this shape, `search`, `insert`, and `delete` all degrade to $O(n)$, because reaching the bottom means visiting every node.

The AVL Tree avoids this entirely. For the exact same sorted input, the AVL detects imbalance after each insert (a balance factor of +2 or −2) and performs rotations to restore balance. The result is a height of about 10 — roughly $log_2(512) \approx 9$ — instead of 512. Its operations stay $O(logn)$ no matter how the data is ordered.

On random data the gap is smaller but still real. A randomly built BST is usually reasonably distributed (height ≈ 19 here), but it is still noticeably taller than the AVL (≈ 11) and offers no guarantee. Therefore, a particularly unlucky ordering could still produce a tall tree.

The AVL pays for its guarantee with extra work on every insert and delete; it tracks each node's height and may perform rotations. A plain BST has simpler, slightly faster individual operations. Ultimately, a BST is best when the input is known to be random and balance is not critical. But when input order is unpredictable — or could be sorted — the AVL Tree is the safer choice because it guarantees logarithmic height and therefore predictable performance.

Appendix

The complete, unabridged console output (all heights and full traversals for every task) is in `output.txt`.